

SRM Institute of Science and Technology

Tiruchirappalli – 621 105

School of Computing

Artificial Intelligence and Machine Learning

Comparative Analysis of ε-Greedy and UCB Algorithms for Sequential Decision Making

PROJECT REPORT

Main Topic: Action Selection

Registration No: RA2311026050020

Student Name: PARVATHY O K

Section: B

Code / Subject Name

21CSE417TA / Fundamentals of Reinforcement Learning

Student Signature **Faculty Signature**

Abstract

This project examines how ε-greedy and Upper Confidence Bound action selection balance learning and reward in a ten-armed bandit. Both methods estimate an action's value from the rewards observed when that action is chosen. Their difference lies in exploration: ε-greedy occasionally chooses an action uniformly at random, whereas UCB1 adds a count-dependent uncertainty bonus to each estimated value.

The study uses reproducible Bernoulli simulations with 500 independent runs of 10,000 decisions per scenario. Two stationary environments test well-separated and closely matched reward probabilities. A third environment reverses the reward probabilities after 5,000 decisions. Uniform random selection and greedy selection provide baselines. A separate parameter study uses 300 runs of 5,000 decisions for each environment. All primary policies receive the same initialization and use sample-mean value estimates.

With well-separated rewards, UCB1 records mean cumulative pseudo-regret of 359.4, compared with 398.4 for ε-greedy at ε = 0.10. With closely matched rewards, the ranking reverses: ε-greedy records 184.0, compared with 399.9 for UCB1. In the reversal scenario, UCB1 records 415.4, while sample-mean ε-greedy records 2,930.1. Replacing the latter's sample mean with a constant step size of 0.10 reduces its regret to 637.8. These values describe the specified simulations and do not establish a universal ranking.

The main conclusion is that action selection must be evaluated together with the reward gaps, decision horizon and value-update rule. UCB1 allocates exploration according to uncertainty and performs well when useful gaps can be resolved. Fixed ε-greedy continues to pay for uniform exploration, but can outperform the standard UCB1 coefficient over a finite horizon when the actions have similar rewards. The report presents the mathematical formulation, implementation, verification and empirical evidence supporting these conclusions.

Keywords: action selection; multi-armed bandit; ε-greedy; UCB1; exploration; pseudo-regret.

Study at a glance

Item	Setting
Primary comparison	ε = 0.10 versus UCB1 with c = √2
Reward model	Bernoulli rewards in {0, 1}; ten actions
Main evaluation	Three scenarios; 500 runs; 10,000 rounds
Statistical reporting	Mean and approximate 95% Monte Carlo interval
Verification	11 executed correctness checks; all passed

Table of Contents

Section / chapter	Page
Abstract	2
Chapter 1 Introduction	**4**
1.1 Decision setting 1.2 Exploration and exploitation 1.3 Scope	4
Chapter 2 Literature Review	**5**
2.1 Foundations 2.2 Confidence methods 2.3 Changing rewards	5
Chapter 3 Problem Statement and Objectives	**6**
Chapter 4 System Analysis and Requirements	**7**
Chapter 5 Methodology and System Design	**8**
5.1 Bandit model 5.2 Value estimates 5.3 Action-selection rules	8
5.4 Algorithm flow 5.5 Exploration behaviour	9
5.6 Experimental scenarios and controls	10
5.7 Evaluation measures and uncertainty	11
Chapter 6 Implementation	**12**
6.1 Simulation components 6.2 Randomness and reproducibility	12
6.4 Worked decision 6.5 Updates 6.6 Computational cost	13
Chapter 7 Results and Discussion	**14**
7.1 Well-separated rewards	14
7.2 Closely matched rewards	15
7.3 Reversal of the best action	16
7.4 Parameter sensitivity	17
7.5 Allocation patterns and interpretation	18
Chapter 8 Testing and Verification	**19**
Chapter 9 Conclusion and Future Work	**20**
References	21
Appendix A Reproduction Protocol	22

Figures: decision loop (4), experiment pipeline (7), algorithm flowcharts (9), reward rates (10), stationary performance (14–15), response to change (16), parameter sensitivity (17), action allocation (18).

Chapter 1 Introduction

1.1 Sequential decisions with incomplete feedback

A system often has to choose an action before knowing which option is best. A recommendation service may select one item to display, or a communication system may choose one available channel. The system observes the outcome of the action it takes; it does not automatically learn what every other action would have produced. Repeating this decision creates a trade-off between collecting information and using the information already available.

A multi-armed bandit isolates this action-selection problem. An action is called an arm, and its reward is a random observation with an unknown mean. In the stationary version, an action's reward distribution stays fixed. Unlike a general Markov decision process, this experiment has no evolving state controlled by the chosen action and no delayed reward chain. Learning still affects later decisions because the agent's estimates change with experience [1].

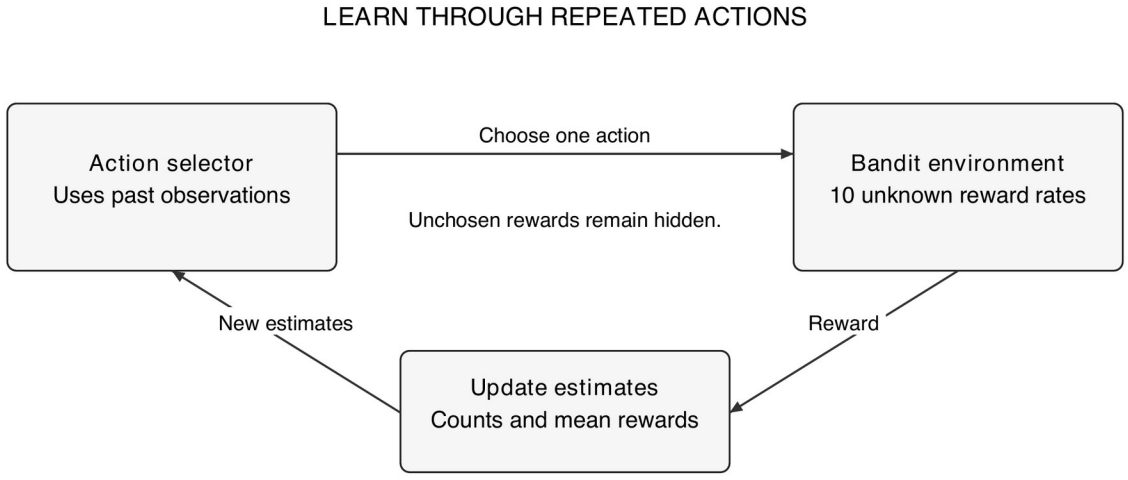


Figure 1.1 The action-selection loop and the feedback available to the learner.

1.2 Exploration and exploitation

Exploitation selects an action with the highest current estimated reward. Exploration samples alternatives that may be better than their present estimates suggest. A high estimate based on one success is less reliable than the same estimate based on hundreds of observations. An effective policy must decide both when to explore and which action deserves another observation.

1.3 Scope of the project

This report compares fixed ε-greedy with UCB1 under controlled synthetic rewards. It examines cumulative reward, pseudo-regret, best-action selection and action counts. The experiments include two stationary settings and one abrupt change. They do not model user context, delayed feedback, costs that differ by action, or a deployed application. This scope keeps the comparison focused on action selection rather than on unrelated software features.

Chapter 2 Literature Review

2.1 Foundations of action-value learning

Sutton and Barto [1] introduce bandits as a setting for studying evaluative feedback and action-value methods. Their treatment explains greedy selection, randomized exploration and upper-confidence action selection. It also distinguishes sample-average estimation from recency-weighted estimation when rewards change. These ideas motivate the common value estimator used in the primary comparison and the constant-step diagnostic used after the reward reversal.

2.2 Finite-time confidence methods

Auer, Cesa-Bianchi and Fischer [2] analyse UCB1 for independent stationary rewards bounded in [0, 1]. Their result gives a gap-dependent logarithmic upper bound on expected regret. The guarantee concerns growth with the horizon under stated assumptions; it does not imply that UCB1 will outperform every alternative in a short experiment. The present study therefore separates the standard coefficient from smaller coefficients tested only in the sensitivity analysis.

Bubeck and Cesa-Bianchi [3] review regret analysis for stochastic and nonstochastic bandits. Their framework distinguishes different feedback and reward assumptions. This distinction matters here: the principal experiments are stochastic, whereas the reversal scenario violates stationarity. A stationary guarantee cannot be transferred to that scenario simply because the same policy is used.

2.3 Changing rewards and further confidence methods

Garivier and Moulines [4] study discounted and sliding-window UCB policies for switching bandits. These methods reduce the influence of old observations. They provide a useful direction for extending the present reversal experiment; neither method is included in the numerical ranking reported here.

Garivier and Cappé [5] develop KL-UCB, which uses an information-based confidence construction. Its treatment of bounded rewards suggests a more specific comparator for Bernoulli observations. This report retains UCB1 as the primary confidence method so that its count-based exploration term can be inspected directly.

Table 2.1 Literature used to define and interpret the comparison.

Source	Relevance to this report
Sutton and Barto [1]	Action values, exploration and value updates
Auer et al. [2]	UCB1 rule and stationary regret guarantee
Bubeck and Cesa-Bianchi [3]	Regret definitions and model assumptions
Garivier and Moulines [4]	Forgetting old observations after a change
Garivier and Cappé [5]	A possible Bernoulli-specific extension

2.4 Purpose of the comparison

The contribution is a transparent comparative experiment, not a new bandit algorithm. Keeping the environment, initialization and evaluation consistent makes it possible to explain observed differences through the selection and update rules. Reporting scenarios in which the ranking changes is part of that comparison.

Chapter 3 Problem Statement and Objectives

3.1 Problem statement

Given ten actions with unknown reward probabilities and a fixed decision budget, select one action at each round to obtain as much reward as possible. The learner sees only the selected reward. It must estimate action quality from incomplete and noisy evidence while spending part of its budget on exploration. The comparison asks how the exploration mechanism changes the total cost of selecting suboptimal actions.

The primary policies are ε-greedy with $\varepsilon = 0.10$ and UCB1 with its standard exploration coefficient. Both start with zero estimates, sample every action once, and then update the selected estimate using the sample mean. Uniform random and purely greedy selection establish the cost of exploring without learning and exploiting without continued exploration.

3.2 Research questions

The first question is whether uncertainty-based exploration improves cumulative performance when one action has a clear advantage. The second is whether that advantage persists when reward probabilities are close. The third is how the policies respond when the best action changes after substantial learning. A final question concerns the sensitivity of finite-horizon results to the exploration parameter.

3.3 Project objectives

1. Define a bounded stochastic bandit model and implement the two selection rules without giving the policies access to the true reward means.

2. Use identical initialization, reward generation and run counts so that differences in measured performance can be interpreted.

3. Measure reward, cumulative pseudo-regret, best-action frequency and action allocation rather than relying on a single learning curve.

4. Quantify variation across independent runs and report paired differences for the primary comparison.

5. Check the update rules, tie handling, initialization, random seeds and regret calculation with executable tests.

6. Identify the limits of the findings and propose extensions that address those limits.

3.4 Assumptions and acceptance criteria

Rewards are immediate and binary. Actions have equal cost, and each scenario has a known, fixed number of actions. Within each stationary segment, reward draws are independent. The simulator knows the reward probabilities for evaluation, while the policy receives only its own estimates, counts and round number. A complete run must contain exactly one action and one update per round, produce finite estimates, and reproduce its output under the same seed and software versions.

Chapter 4 System Analysis and Requirements

4.1 Experimental system

The implementation is a local numerical experiment. Configuration defines the environment and policies. A reward generator creates potential outcomes, the runner exposes only the chosen outcome, and a separate evaluator computes performance. The separation in Figure 4.1 prevents the true means from influencing action selection.

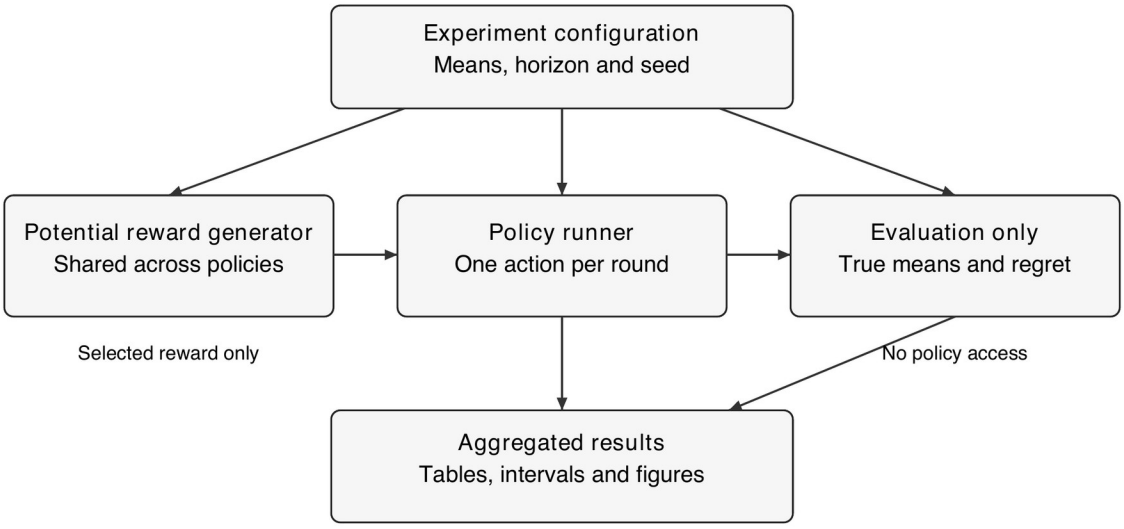


Figure 4.1 Simulation components and the boundary between learning and evaluation.

4.2 Functional requirements

Table 4.1 Responsibilities of the experimental components.

Component	Required behaviour
Environment	Generate independent Bernoulli outcomes; apply a change only at the specified round.
Policy	Choose one valid action from current estimates, counts and round number.
Estimator	Update only the selected action; preserve all other estimates.
Evaluator	Accumulate reward, pseudo-regret, optimal selections and action counts.
Aggregator	Retain run-level totals and compute means and uncertainty intervals.

4.3 Software and resource requirements

The executed study uses Python 3.12.14 and NumPy 2.3.5. Figures are produced as vector drawings and inserted at 300 dpi. A CPU is sufficient; no GPU, network service or external dataset is required. The runner stores one reward matrix per round instead of all potential outcomes across the full horizon. This keeps memory use proportional to the number of runs, actions and policies rather than to the full simulation history.

Chapter 5 Methodology and System Design

5.1 Bandit model and notation

Let K = 10 be the number of actions and T the number of rounds. At round t, the learner selects A_t and receives $R_t \in \{0, 1\}$. The reward probability of action a is μ_a. The best mean is $\mu^* = \max_a \mu_a$, and the gap of action a is $\Delta_a = \mu^* - \mu_a$. Counts and estimates used for selection contain observations strictly before round t.

$$R_t \mid A_t = a \quad \sim \quad Bernoulli(\mu_a) \quad (5.1)$$

Table 5.1 Notation used in the algorithm descriptions.

Symbol	Meaning
K, T	Number of actions and decision horizon
A_t, R_t	Action and observed reward at round t
$N_t(a)$, $Q_t(a)$	Count and reward estimate before round t
μ_a, μ^*, Δ_a	Action mean, best mean and suboptimality gap
ε, c, α	Random exploration rate, UCB coefficient and update step size

5.2 Incremental value estimation

After choosing action a and observing a reward, its count increases by one. The new estimate is obtained from the previous estimate and the prediction error. Unselected actions are unchanged. This update equals the arithmetic mean of all rewards observed for that action and avoids storing its complete reward history.

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N_t(a) + 1}[R_t - Q_t(a)] \quad (5.2)$$

5.3 Action-selection rules

After the first ten initialization rounds, ε-greedy chooses a uniformly random action with probability ε and a largest-estimate action otherwise. Random exploration includes the greedy action. If the greedy action is unique, its selection probability is 1 − ε + ε/K; every other action has probability ε/K. Equal estimates are resolved uniformly at random.

$$A_t = arg \ max_a [Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}}] \quad (5.3)$$

Equation (5.3) defines the UCB family used here. UCB1 sets c = √2. The count-dependent term is larger for actions with fewer observations. Every action is sampled once before this expression is used, so its denominator is positive. A UCB score can exceed one; it is an optimistic index and is not clipped to the reward range.

Chapter 5 Methodology and System Design

5.4 Algorithm flow

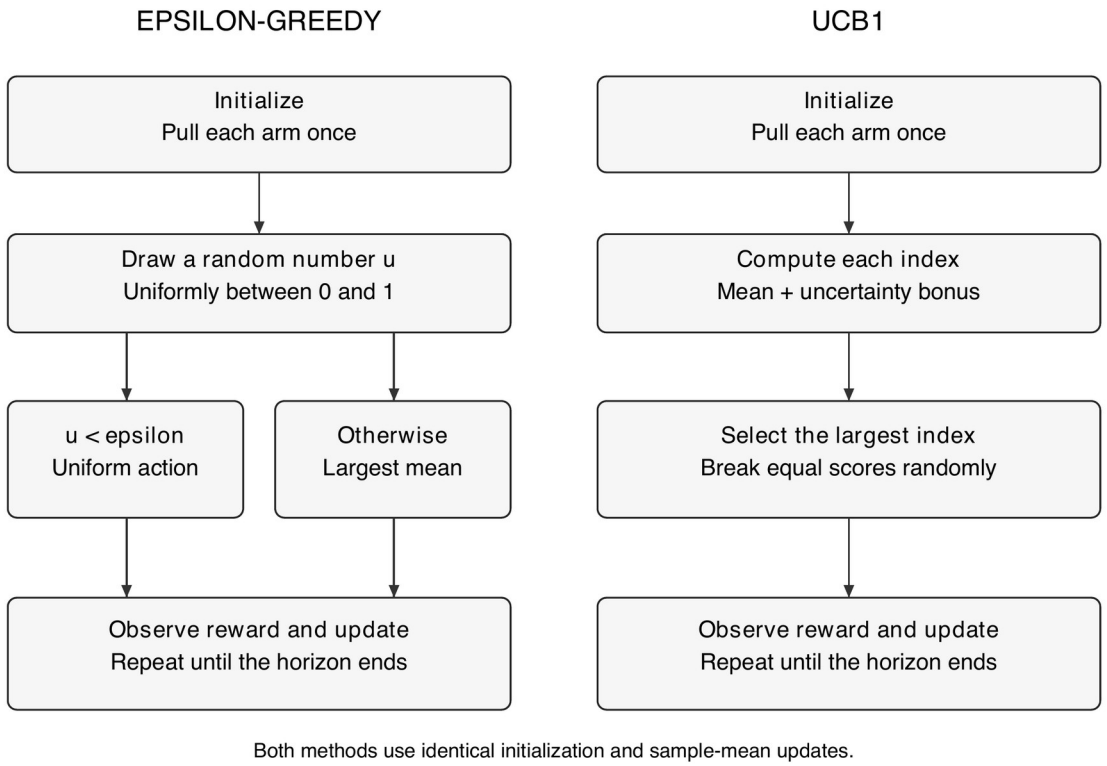


Figure 5.1 The two selection rules share initialization, feedback and estimation.

5.5 Expected exploration behaviour

For fixed ε, exploration continues even after the correct action has been identified. In a stationary environment, the expected regret contributed by the random branch after initialization is ε/K times the sum of the action gaps per round. Additional regret occurs whenever the greedy branch selects an incorrect action. In the well-separated scenario, ε = 0.10 therefore incurs a continuing random-exploration cost of 0.036 reward units per round.

UCB1 does not reserve a fixed fraction of decisions for uniform exploration. Its bonus decreases as an action accumulates observations and increases slowly with the round number. Once evidence separates the actions, the policy can spend more decisions on the best one. For stationary independent bounded rewards with fixed positive gaps, its established expected-regret bound grows logarithmically with the horizon [2]. That guarantee does not specify the winner at T = 10,000, particularly when gaps are small.

The parameter experiment also tests $\varepsilon_t = \min(1, 1/\sqrt{t})$. This is an illustrative decreasing schedule, not a claim of a tuned or universally optimal decay rule. All primary comparisons retain their parameters throughout a run.

Chapter 5 Methodology and System Design

5.6 Experimental scenarios and controls

Each main scenario contains 500 independent runs of 10,000 rounds. Actions are numbered 1 through 10. Rewards are sampled from the probabilities in Table 5.2. The two stationary scenarios differ in the gap between the best and second-best action: 0.20 in the well-separated case and 0.02 in the closely matched case.

Table 5.2 Exact Bernoulli reward probabilities.

Action	Well-separated	Closely matched	After reversal
1	0.10	0.40	0.70
2	0.15	0.41	0.50
3	0.20	0.42	0.45
4	0.25	0.43	0.40
5	0.30	0.44	0.35
6	0.35	0.45	0.30
7	0.40	0.46	0.25
8	0.45	0.47	0.20
9	0.50	0.48	0.15
10	0.70	0.50	0.10

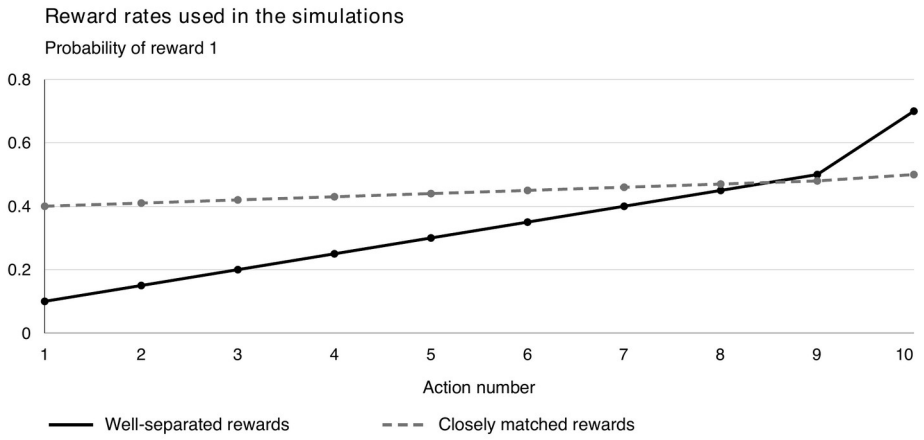


Figure 5.2 Reward profiles used in the two stationary scenarios.

In the reversal scenario, the well-separated vector applies through round 5,000 and its reverse applies from round 5,001. Action 10 is initially best; action 1 becomes best. The best achievable mean remains 0.70. Policies are not told about the change, and their estimates are not reset.

All policies sample actions 1 to 10 once before normal selection. Within a run and round, they share the same potential reward for each action, but each sees only its chosen reward. Policy randomization uses separate streams. Runs are independent, and environment seeds differ across scenarios. This design supports paired comparisons without exposing unchosen rewards.

Chapter 5 Methodology and System Design

5.7 Evaluation measures

Mean reward per round is total observed reward divided by the horizon. It is directly interpretable for binary rewards as the fraction of decisions receiving reward one. Pseudo-regret measures the expected opportunity cost of the selected actions using the known simulation means; it removes the extra variation caused by the realised reward draw.

$$L^{(i)}(T) = \sum_{t=1}^{T} [\mu^* - \mu_{A_t}] \quad (5.4)$$

Here i denotes a run. The reported stationary pseudo-regret is the mean of $L^{(i)}(T)$ across runs. It is nonnegative for every action sequence. It differs from realised regret $T\mu^* - \Sigma R_t$, which contains reward noise and can occasionally be negative. In the changing environment, the evaluator uses the best mean and chosen-action mean at the current round, giving dynamic pseudo-regret.

$$L^{(i)}_{dynamic}(T) = \sum_{t=1}^{T} [\mu^*_t - \mu_t(A_t)] \quad (5.5)$$

Best-action selection is the proportion of rounds assigned to the action with the largest true mean. The report also gives this proportion over the last 1,000 rounds. Action counts reveal where exploration is spent. Reward curves use non-overlapping 100-round blocks; regret curves show cumulative values at those same checkpoints. Blocking is a plotting choice and does not change any totals.

5.8 Uncertainty and paired comparisons

For a run-level measure X, the report gives its mean with an approximate 95% Monte Carlo interval. The sample standard deviation is computed across independent runs with the usual n − 1 denominator. The interval describes simulation variation for the fixed scenario, not uncertainty over all possible bandit environments.

$$Interval = \overline{X} \pm 1.96 \frac{s}{\sqrt{n}} \quad (5.6)$$

Primary policy differences use $D_i = L_i(\text{UCB1}) - L_i(\varepsilon\text{-greedy})$ within each matched run, followed by the same interval calculation on D. Negative values favour UCB1. Bands on learning curves are pointwise intervals, not simultaneous bands. Parameter sweeps are exploratory and are not adjusted for multiple comparisons; selecting a coefficient from them would require a separate validation study.

5.9 A check on interpretation

In the well-separated environment, uniform selection has expected mean reward 0.34 and pseudo-regret 10,000 × (0.70 − 0.34) = 3,600. The measured mean regret is 3,601.0. Agreement with this analytic baseline provides a direct check on the environment and evaluation logic.

Chapter 6 Implementation

6.1 Simulation components

The implementation stores one row per independent run and one column per action. For each policy, Q holds floating-point estimates and N holds integer counts. A round is processed for all runs together using NumPy arrays. This vectorization changes execution speed but does not allow information to pass between runs.

Table 6.1 Responsibilities of the implemented routines.

Component	Implementation responsibility
choose	Applies forced initialization, random selection, ε-greedy or the UCB index.
update	Increments selected counts and updates only selected estimates.
experiment	Generates rewards, advances policies and accumulates evaluation measures.
ci	Calculates a run-level mean and its Monte Carlo interval half-width.
Figure builder	Reads saved results and draws diagrams, curves and uncertainty intervals.

6.2 Randomness and reproducibility

The environment generator uses NumPy default_rng with SeedSequence([seed, 0]). Policy j uses SeedSequence([seed, j + 1]), where j starts at zero. The main policy order is uniform random, greedy, ε-greedy and UCB1. The constant-step ε-greedy variant is appended only in the reversal scenario. A shared matrix of independent uniform numbers is converted into potential Bernoulli rewards once per round.

```
for t in 1 .. T:
    means = current_reward_probabilities(t)
    outcomes = environment_rng.random((runs, K)) < means
    for policy in policies:
        action = choose(Q, N, t, policy, policy_rng)
        reward = outcomes[run_index, action]
        update_selected_estimate(Q, N, action, reward)
        accumulate_reward_and_regret(action, reward, means)
```

The policy routine never receives means or the full outcomes matrix. Evaluation happens after the action has been selected. Exact estimate ties use independent random priorities restricted to the tied actions; numerical perturbations are not added to unequal values.

6.3 Recorded outputs

For each policy and scenario, the run records total reward, cumulative pseudo-regret, optimal-selection count, last-1,000-round optimal selections, post-change regret and final action counts. Checkpoints retain 100-round reward and optimal-selection measures. Run-level regret totals are retained for paired comparisons, so the uncertainty of a difference is calculated directly rather than inferred from two separate intervals.

The accompanying epsilon-greedy-ucb repository includes a local browser dashboard. It displays the saved study and accepts new simulation settings. These additional comparisons do not overwrite the report data. The repository also contains the numerical code, tests, CSV exports, figure sources and report generator.

Chapter 6 Implementation

6.4 Worked action-selection example

Consider a three-action illustration at round 100. The counts sum to 99, and the estimates shown in Table 6.2 are the information available before the next decision. The illustration is separate from the ten-action experiments and is used only to make the selection calculation explicit.

Table 6.2 A calculation showing how uncertainty changes a decision.

Action	Estimate Q	Count N	UCB1 bonus	UCB1 score
1	0.629	70	0.363	0.991
2	0.600	25	0.607	1.207
3	0.500	4	1.517	2.017

Greedy selection chooses action 1 because it has the largest current estimate. With ε = 0.10, ε-greedy chooses action 1 with probability 0.90 + 0.10/3 = 0.9333 and each alternative with probability 0.0333. UCB1 chooses action 3 because its uncertainty bonus outweighs its lower estimate. This does not mean action 3 is known to be better; it means another observation is valuable under the UCB rule.

6.5 Updating an estimate

The estimates correspond to 44, 15 and 2 successes respectively. If action 3 produces reward one, its count becomes five. Its sample-mean estimate changes from 0.50 to 0.50 + (1 − 0.50)/5 = 0.60. The other two estimates remain unchanged. This is the same update used in the primary simulation.

For the constant-step diagnostic, the update is $Q \leftarrow Q + \alpha(R - Q)$, with $\alpha = 0.10$. A reward of one changes an estimate of 0.50 to 0.55 regardless of how many past rewards were observed. This places more weight on recent outcomes than a sample mean after many selections, at the cost of persistent variation under stationary rewards.

6.6 Computational cost

Both implemented primary rules scan all K action values, giving O(K) selection time per round and O(K) state per run. Updating one selected action takes O(1). UCB1 additionally evaluates a square root for each index. Uniform exploration itself is inexpensive, but finding and resolving a greedy maximum still requires a scan in this implementation.

For n runs and P policies, the main arrays require O(nKP) storage. The temporary potential-reward matrix requires O(nK). Checkpoint summaries require O(PT/100) storage, and the run-level totals require O(nP). No timing comparison is used to rank the policies because language overhead and vectorization would confound a simple runtime claim.

Chapter 7 Results and Discussion

7.1 Well-separated rewards

The first scenario has a best reward probability of 0.70 and a second-best probability of 0.50. UCB1 has lower final pseudo-regret than ε-greedy at ε = 0.10. The paired difference is −39.0 ± 6.4 reward units, an approximate 95% Monte Carlo interval. This is a 9.8% reduction relative to the ε-greedy mean in this scenario.

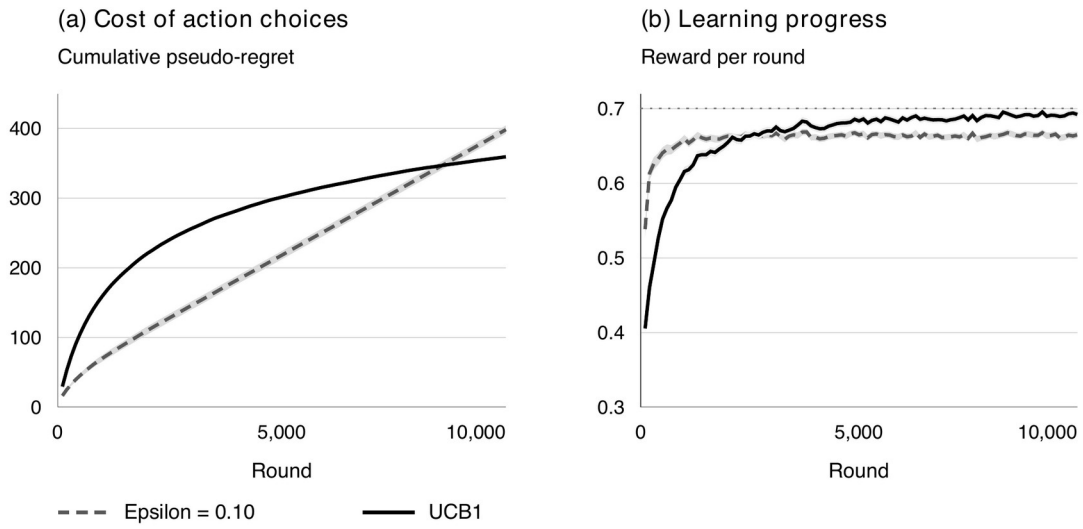


Figure 7.1 Well-separated rewards. Shading shows pointwise 95% intervals; reward uses 100-round blocks. The dotted line marks the best mean reward.

Table 7.1 Well-separated rewards over 10,000 rounds. Values use mean ± 95% interval half-width.

Policy	Mean reward	Pseudo-regret	Best action last 1,000
Uniform random	0.3399 ± 0.0004	3601.0 ± 1.5	10.0%
Greedy	0.6089 ± 0.0115	914.1 ± 115.0	65.0%
ε-greedy, ε = 0.10	0.6601 ± 0.0007	398.4 ± 5.9	90.9%
UCB1	0.6639 ± 0.0005	359.4 ± 2.4	97.1%

UCB1 spends more decisions on exploration early in the run, but its regret curve becomes flatter as the evidence improves. During the last 1,000 rounds it selects the best action 97.1% of the time, compared with 90.9% for ε-greedy. The latter value is close to $1 - \varepsilon + \varepsilon/K = 0.91$, the probability obtained when its greedy choice is correct.

The full-horizon best-action proportions are 88.8% for UCB1 and 89.4% for ε-greedy. This apparent reversal is consistent with lower regret for UCB1: regret depends on which suboptimal actions are selected, not only on the number of nonoptimal decisions. Greedy selection has a much wider interval because some runs commit to an inferior action after early outcomes.

Chapter 7 Results and Discussion

7.2 Closely matched rewards

The second scenario has reward probabilities between 0.40 and 0.50 and a best-versus-second-best gap of only 0.02. At the same horizon, ε-greedy has substantially lower pseudo-regret than standard UCB1. The paired UCB1-minus-ε-greedy difference is 215.9 ± 10.2. Here the ε-greedy mean regret is 54.0% below the UCB1 mean.

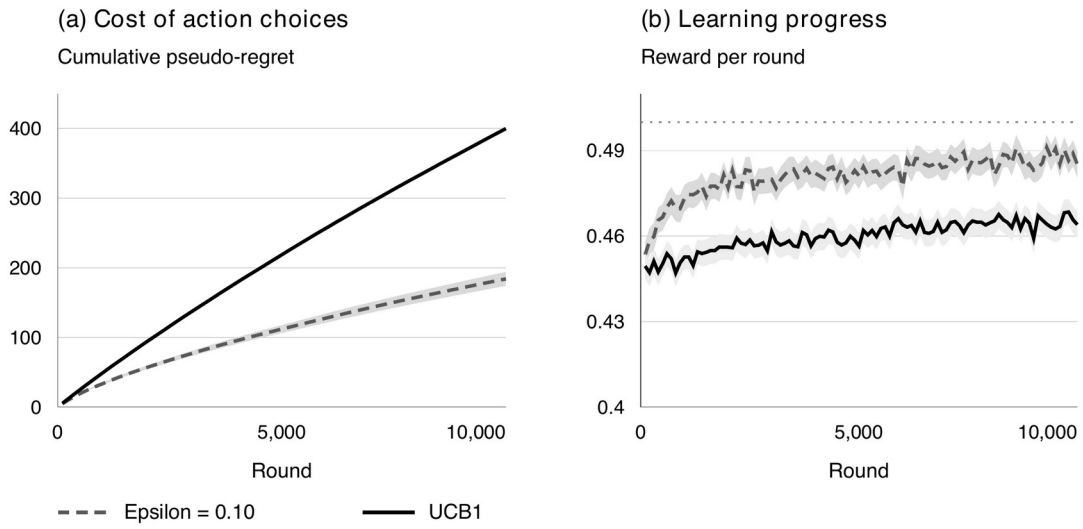


Figure 7.2 Closely matched rewards under the same horizon and run count. Shading shows pointwise 95% intervals; the dotted line marks the best mean.

Table 7.2 Closely matched rewards over 10,000 rounds.

Policy	Mean reward	Pseudo-regret	Best action last 1,000
Uniform random	0.4459 ± 0.0004	540.1 ± 0.3	10.1%
Greedy	0.4619 ± 0.0024	379.9 ± 24.1	20.2%
ε-greedy, ε = 0.10	0.4813 ± 0.0011	184.0 ± 10.0	60.9%
UCB1	0.4598 ± 0.0005	399.9 ± 1.9	28.0%

UCB1 continues to allocate many decisions to competing actions because differences in their estimates remain small relative to the confidence bonuses. The policy is cautious about eliminating actions, which limits its best-action frequency during this finite experiment. ε-greedy concentrates its nonrandom decisions more strongly, although it also fails to identify the best action in some runs.

A low best-action frequency need not imply a large reward deficit when the alternatives are close. For this reason, pseudo-regret and mean reward are more informative than best-action frequency alone. The result does not contradict the stationary logarithmic guarantee: a bound on long-run growth does not impose an ordering between algorithms at every finite horizon or for every gap configuration.

Chapter 7 Results and Discussion

7.3 Reversal of the best action

At round 5,001, all reward probabilities are reversed. UCB1 has the lowest dynamic pseudo-regret among the tested policies in this particular change pattern. Sample-mean ε-greedy is slow to recover, while the constant-step variant responds more effectively. Figure 7.3 measures selection of the action that is best at the current round.

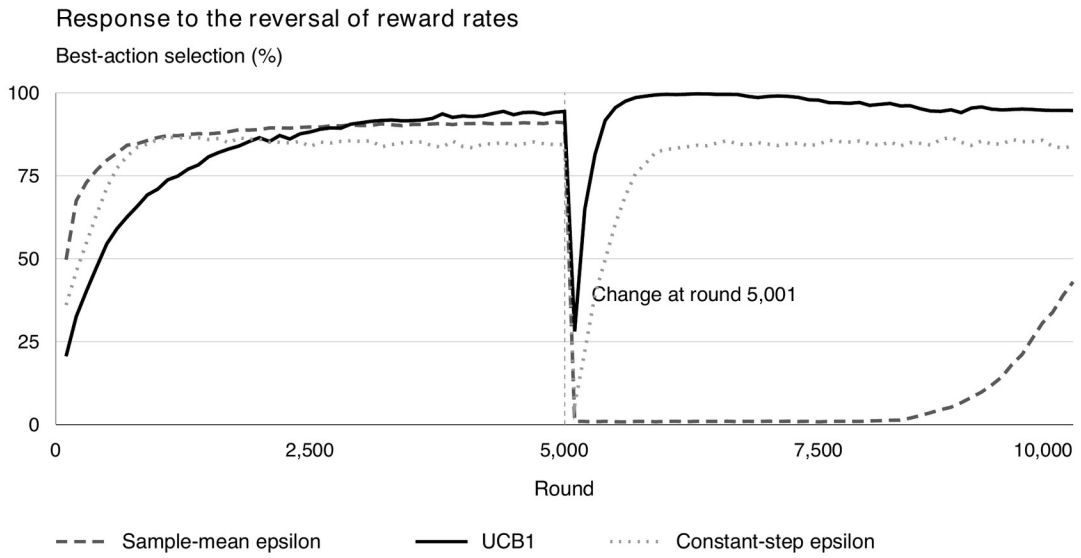


Figure 7.3 Best-action selection in non-overlapping 100-round blocks before and after the reversal. Each curve is a mean over 500 runs.

Table 7.3 Reversal scenario over 10,000 rounds.

Policy	Mean reward	Dynamic regret	Best action last 1,000
Uniform random	0.3399 ± 0.0004	3599.0 ± 1.4	10.0%
Greedy	0.3838 ± 0.0042	3162.3 ± 42.5	4.6%
ε-greedy, ε = 0.10	0.4070 ± 0.0020	2930.1 ± 20.2	24.8%
UCB1	0.6583 ± 0.0007	415.4 ± 5.5	95.0%
ε-greedy, α = 0.10	0.6362 ± 0.0010	637.8 ± 8.9	84.8%

The measured post-change regret is 2,705.8 for sample-mean ε-greedy, 114.0 for UCB1 and 344.2 for constant-step ε-greedy. Old observations dominate the sample mean of frequently selected actions. Meanwhile, ε-greedy revisits a specific alternative through its random branch on only about ε/K = 0.01 of decisions, delaying the recognition of a previously poor action.

UCB1 repeatedly revisits actions with lower counts. In this reversal, that behaviour helps it discover the newly strong action while evidence against the old best action accumulates. This is an interpretation of the observed curves and counts, not a general adaptation guarantee. Other change times or reward patterns may produce different results. Discounted or sliding-window methods require a separate comparison [4].

Chapter 7 Results and Discussion

7.4 Parameter sensitivity

A separate study uses 300 runs and 5,000 rounds per stationary scenario. It varies ε while retaining sample-mean updates, and varies c in Q + c√(ln t/N). The main experiment's parameters remain fixed. Figure 7.4 shows why choosing an exploration setting without reference to the environment can be misleading.

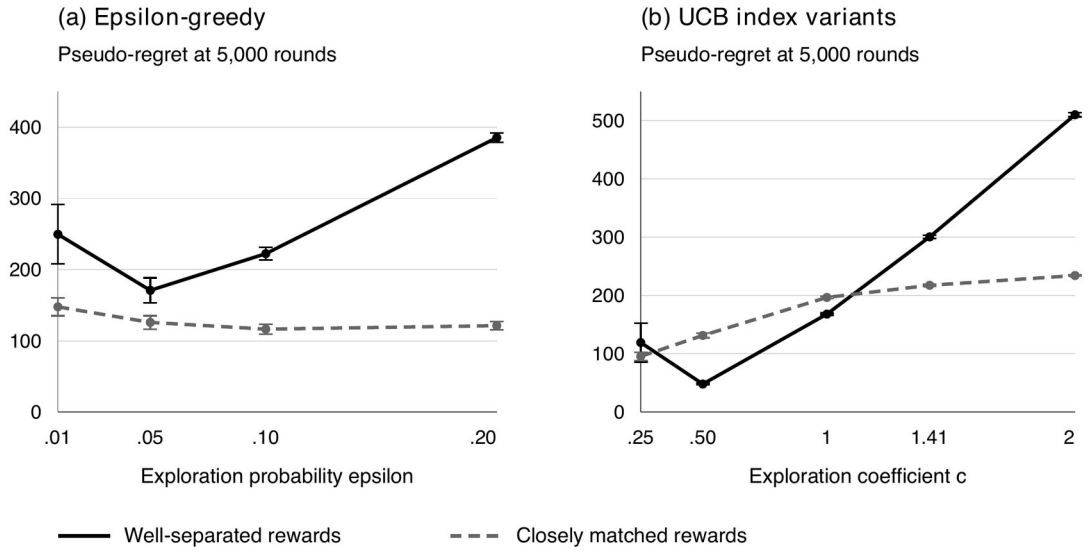


Figure 7.4 Exploration settings and final pseudo-regret. Error bars are approximate 95% Monte Carlo intervals. The standard UCB1 coefficient is c = √2 ≈ 1.414.

Table 7.4 Selected sensitivity results at the shorter 5,000-round horizon.

Setting	Well-separated regret	Closely matched regret
ε = 0.01	249.6 ± 41.6	147.8 ± 12.6
ε = 0.05	170.9 ± 17.5	126.0 ± 9.4
ε = 0.10	222.4 ± 8.7	116.5 ± 7.0
ε = 0.20	385.2 ± 6.7	121.3 ± 5.9
ε_t = min(1, 1/√t)	134.0 ± 25.2	124.3 ± 10.7
UCB c = 0.50	48.2 ± 1.4	131.6 ± 3.9
UCB1 c = √2	300.6 ± 2.8	217.6 ± 1.4

Very small ε reduces continued exploration but increases the risk of an early poor commitment. Larger ε explores more reliably but pays a continuing cost. For UCB, smaller c can improve finite-horizon performance here; the best plotted coefficient differs between the two reward profiles. Such variants are not the standard UCB1 policy and do not inherit its stated coefficient-specific guarantee. These exploratory results are not an independently validated tuning recommendation.

Chapter 7 Results and Discussion

7.5 Action allocation explains the regret difference

Figure 7.5 shows average selections of the nine suboptimal actions in the well-separated environment. The best action is omitted to make the smaller counts readable. The action probabilities increase from action 1 to action 10, so actions near the right edge are stronger competitors.

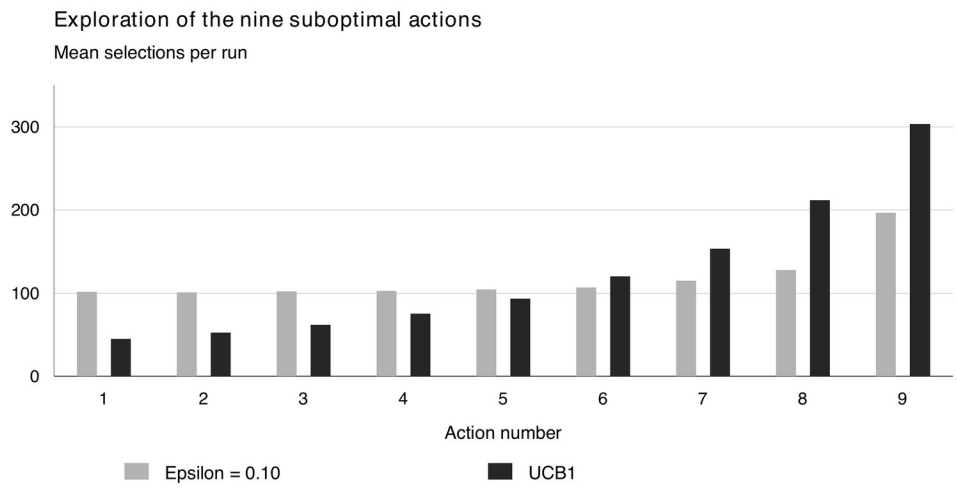


Figure 7.5 Mean selections of suboptimal actions over 10,000 rounds.

Uniform exploration revisits every action regardless of how clearly it has been outperformed. UCB1 concentrates more visits on competitive alternatives. A near-best action and a poor action count equally as nonoptimal decisions but incur different regret, explaining why best-action frequency alone can misrepresent performance.

7.6 Comparison across the three scenarios

Table 7.5 Paired differences across 500 runs at T = 10,000.

Scenario	UCB1 minus ε-greedy regret	Lower mean regret
Well-separated	−39.0 ± 6.4	UCB1
Closely matched	215.9 ± 10.2	ε-greedy
Reversal	−2,514.8 ± 20.7	UCB1

The standard confidence bonus is useful in the well-separated and specified reversal environments, but explores too broadly to beat ε-greedy within the closely matched experiment’s budget. Parameter and update choices depend on the problem.

7.7 Limits of the evidence

The reward vectors are fixed and synthetic; the intervals do not cover variation across real applications. The study tests one action count, two main reward-gap profiles and one change pattern. The reversal is not representative of every nonstationary process. No user context, delayed outcomes, action costs or adversarial reward generation is included. Broader claims would require more environments and a separate tuning and evaluation protocol.

Chapter 8 Testing and Verification

8.1 Executed correctness checks

Eleven automated checks were executed before the report was assembled. They test calculations with known answers and behaviour at important boundaries. All eleven passed. The simulation also checks that every run contains exactly T selections and that its value estimates remain finite and within [0, 1].

Table 8.1 Executed algorithm and evaluation checks.

ID	Check	Expected evidence	Result
T01	Sample-mean update	Rewards 1, 0, 1, 1 give estimate 0.75.	PASS
T02	UCB uncertainty	A sparsely sampled action wins the known index comparison.	PASS
T03	Initialization	Every action has count one after K rounds.	PASS
T04	ε = 0	Every selected action is the unique greedy maximum.	PASS
T05	ε = 1	30,000 selections are within 0.015 of uniform proportions.	PASS
T06	Equal-value ties	No first-action bias within the same frequency tolerance.	PASS
T07	Constant step size	Q = 0.50, R = 1, α = 0.10 gives Q = 0.55.	PASS
T08	Equal true means	Pseudo-regret is exactly zero.	PASS
T09	Seed reproducibility	Repeated runs with the same seed match exactly.	PASS
T10	Interval calculation	Mean and standard error match a hand calculation.	PASS
T11	Dynamic benchmark	A two-round changing optimum yields zero regret when followed.	PASS

8.2 Integration and analytical checks

The independent-run count is the statistical unit for uncertainty estimates. Checkpoint averages are not treated as additional independent runs. The random baseline agrees with its analytic expectation: its well-separated mean regret is 3,601.0 against an expectation of 3,600, and its closely matched mean regret is 540.1 against an expectation of 540. The last-round counts sum to 10,000 in every main run.

8.3 What the checks establish

These checks support the correctness of the implemented selection, updating and measurement logic for the tested cases. They do not prove a regret theorem or certify performance in a deployed system. The parameter sweep is interpreted descriptively, and the observed reversal performance is kept separate from stationary theoretical guarantees.

Chapter 9 Conclusion and Future Work

9.1 Conclusion

This project compared ε-greedy and UCB1 as action-selection methods in a controlled ten-armed bandit. The comparison held initialization and the main value estimator constant, used shared potential rewards for paired evaluation, and measured outcomes across 500 independent runs per scenario. The results show that a useful comparison must state the reward gaps, horizon, exploration settings and update rule.

UCB1 reduced mean pseudo-regret from 398.4 to 359.4 in the well-separated environment. In the closely matched environment, ε-greedy reduced regret from UCB1's 399.9 to 184.0. The change in ranking is the central finding. Confidence-based exploration can spend too much of a finite budget distinguishing similar actions, even while retaining a sound stationary theoretical basis.

The reversal experiment also exposed a limitation of long-memory estimation. Sample-mean ε-greedy retained outdated evidence and adapted slowly. A constant step size improved its response, while UCB1 performed best among the tested policies for this specific reversal. This result supports further study of forgetting mechanisms rather than a claim that standard UCB1 handles arbitrary change.

9.2 Practical implications within the study scope

For a stationary task with a clearly superior action, the uncertainty-driven allocation used by UCB1 is a reasonable baseline. For a short decision budget with closely matched actions, the exploration coefficient deserves explicit evaluation. A fixed ε is easy to interpret, but it retains an exploration cost after learning. Decreasing ε can reduce that cost while making premature commitment more consequential.

An action-selection method and a value-update rule should be chosen together. When old evidence may become irrelevant, retaining every past observation with equal weight can delay recovery. Mean reward, regret and allocation counts should be examined together; a best-action percentage alone can conceal how costly the alternative selections are.

9.3 Future work

The first extension is to vary the number of actions, decision horizon and reward gaps across many generated environments. Parameters should be selected on one set of environments and evaluated on a separate set. Longer horizons would help show when the finite-horizon ordering changes.

The second extension is to test repeated and gradual changes using discounted or sliding-window estimates. A third is to compare KL-UCB and Thompson sampling on the same Bernoulli tasks. Contextual rewards, delayed feedback and unequal action costs would move the problem closer to particular applications, but each extension requires a revised model and evaluation protocol.

The report provides a reproducible basis for those extensions: explicit scenarios, original figures, uncertainty estimates and tested implementations. Its conclusions remain limited to the assumptions and experiments reported here.

References

[1] Sutton, R. S., and Barto, A. G. (2018). Reinforcement Learning: An Introduction. 2nd edition. MIT Press. Chapter 2, Multi-armed Bandits.

https://mitpress.mit.edu/9780262039246/reinforcement-learning/

[2] Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002). Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning, 47, 235–256.

https://doi.org/10.1023/A:1013689704352

[3] Bubeck, S., and Cesa-Bianchi, N. (2012). Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems. Foundations and Trends in Machine Learning, 5(1), 1–122.

https://arxiv.org/abs/1204.5721

[4] Garivier, A., and Moulines, E. (2011). On Upper-Confidence Bound Policies for Switching Bandit Problems. Algorithmic Learning Theory, Lecture Notes in Computer Science, 6925, 174–188.

https://doi.org/10.1007/978-3-642-24412-4_16

[5] Garivier, A., and Cappé, O. (2011). The KL-UCB Algorithm for Bounded Stochastic Bandits and Beyond. Proceedings of the 24th Annual Conference on Learning Theory, Proceedings of Machine Learning Research, 19, 359–376.

https://proceedings.mlr.press/v19/garivier11a.html

Relationship between sources and experimental evidence

The cited works provide the algorithmic context and theoretical assumptions. All numerical results in Chapter 7 come from the simulation protocol described in this report. The figures are original drawings or plots of those simulation outputs. Numerical results from unrelated applications are not used as evidence for the present comparison.

UCB1 denotes the coefficient $c = \sqrt{2}$ throughout the main comparison. Other values of c are identified as UCB index variants. The nonstationary results are empirical observations and are not presented as consequences of the stationary regret guarantee.

Appendix A Reproduction Protocol

A.1 Exact experiment settings

Table A.1 Seeds and budgets for the completed experiments.

Experiment	Runs	Rounds	Environment seed
Well-separated	500	10,000	20261111
Closely matched	500	10,000	20261112
Reversal	500	10,000	20261113
Sensitivity, well-separated	300	5,000	20262111
Sensitivity, closely matched	300	5,000	20262112

Use the probabilities in Table 5.2. Initialize Q and N to zero. Force action t in rounds 1 to 10, then apply the policy rule. Main policies use sample-mean updates; only the additional reversal diagnostic uses $\alpha = 0.10$. The change is applied before choosing and rewarding round 5,001, without resetting any policy state.

A.2 Core update and selection pseudocode

```
INITIALIZE Q[a] = 0 and N[a] = 0 for every action a
FOR round t = 1, ..., T:
    IF t <= K:
        a = t
    ELSE IF policy is epsilon-greedy:
        IF Uniform(0, 1) < epsilon:
            a = UniformRandomAction(1, ..., K)
        ELSE:
            a = RandomArgmax(Q)
    ELSE IF policy is UCB1:
        score[a] = Q[a] + sqrt(2 * ln(t) / N[a])
        a = RandomArgmax(score)
    r = reward for the selected action a
    N[a] = N[a] + 1
    Q[a] = Q[a] + (r - Q[a]) / N[a]
    record r, current_best_mean - current_mean[a], a
```

A.3 Sensitivity order and reporting

Sensitivity policies are ordered as $\varepsilon = 0.01, 0.05, 0.10, 0.20$; $\varepsilon_t = \min(1, 1/\sqrt{t})$; then UCB coefficients $c = 0.25, 0.50, 1.00, \sqrt{2}$ and 2.00. Generate the environment and policy streams as described in Section 6.2. Use the same potential-outcome matrix for every policy at each round, retaining only the selected outcome for the learner.

Average results over runs, not over checkpoint summaries. For each interval, use the sample standard deviation with one degree of freedom removed and calculate $1.96s/\sqrt{n}$. For paired comparisons, first subtract policy regrets within each run. Plot regret every 100 rounds and use non-overlapping 100-round reward blocks. Round only for display; retain full precision during aggregation.

Execution environment: Python 3.12.14; NumPy 2.3.5. Random streams use NumPy default_rng and SeedSequence. Exact random draws depend on these generator choices and call order; the protocol fixes both in the implementation.

Project repository (source code, datasets, figures and report): https://github.com/Parvathyok/epsilon-greedy-ucb